

falsify-ml: A Falsification Protocol for Deciding Whether an ML Project Is Worth Pursuing

Saurabh Dixit

2026

Abstract

Preprint / position paper. This is a methods-and-position paper, not an empirical results paper. It presents a protocol and an open-source tool, situates them against existing reproducibility and reporting work, and states honestly what it does not yet establish (an empirical evaluation of the protocol’s hit-rate on known failures — see Section 7). It has not been peer-reviewed.

Most machine-learning tooling helps practitioners *build* models; comparatively little helps them decide whether a model — or the idea behind it — is worth building or worth trusting in the first place. Yet a large fraction of ML failures are decided *before* a single training run: a wrongly framed problem, a leaked train/test split, data that cannot physically support the target precision, no baseline, or economics that never close. We present **falsify-ml**, a ten-stage, model-agnostic protocol that treats every load-bearing ML claim as a *hypothesis to be falsified* rather than a result to be confirmed, and an accompanying open-source tool (pure Python standard library, zero dependencies) that runs the mechanical parts of the protocol and produces a single project-level verdict: **PURSUE / DO NOT PURSUE / INSUFFICIENT EVIDENCE**. The protocol synthesizes ideas from the philosophy of science (Popper’s falsification), pre-registration, and the recent ML reproducibility literature (data-leakage detection; reporting standards) into a *pre-build and trust-gate* workflow with runnable diagnostics for the three stages where automation has the most leverage: leakage/split-hygiene, data-feasibility (statistical power and label-noise ceilings), and value/economics. We position falsify-ml relative to prior work, describe the protocol and tool, work through illustrative cases, and are explicit about what we do *not* claim. Code: <https://github.com/saurabhdxt-labs/falsify-ml> (Apache-2.0).

1 Introduction

A model that scores beautifully on its own benchmark can be detecting nothing real. When the benchmark and the training data are drawn from the same generator, an internal metric measures *consistency with the generator*, not agreement with reality — a closed loop that produces a confident number with no external meaning. This failure mode is neither rare nor new: Kapoor and Narayanan [1] document data leakage across 17 scientific fields affecting 294 papers, and show that when leakage is corrected, the apparent superiority of ML methods frequently disappears.

The expensive property of these failures is *when* they are discovered. A leaked split, an unachievable target given label noise, or a model whose work is actually done by a fallback path can survive months of development and downstream building before anyone measures the claim against independent evidence. The cost is not a bug fix; it is the wasted quarter.

The existing response has largely been *reporting and review standards* applied to *finished* work — most prominently REFORMS [2], a consensus 32-item checklist for what an ML-based-science paper should disclose. These are valuable, but they are oriented toward catching problems at review

time and toward reporting, not toward *deciding, before you build, whether the project can work at all*, and not toward *adversarially attacking your own claim* with runnable diagnostics.

falsify-ml addresses that gap. Its premises are:

1. **A claim is a hypothesis until it is closed.** Every load-bearing assertion (“the model works,” “this beats the baseline,” “the detector detects the thing”) is treated as a hypothesis requiring a precise statement, a pre-registered TRUE shape and FALSE shape, evidence from *outside* the loop that produced the model, and a written verdict — before downstream work depends on it.
2. **Most failures are upstream of evaluation.** Falsification of the model’s headline claim is one stage of ten; the others guard problem framing, data hygiene, data feasibility, baselines, economics, training health, eval design, deployment drift, and stopping.
3. **The useful output is a decision, not an analysis.** The protocol rolls per-stage statuses into one verdict — PURSUE / DO NOT PURSUE / INSUFFICIENT EVIDENCE — usable on a bare idea (before data exists) or on a real model with data.

We make no claim that the individual techniques are novel (they are not; see Section 6). The contribution is the *synthesis* into a pre-build-and-trust lifecycle with a runnable, dependency-free tool, and an explicit calibration that keeps the protocol from degenerating into “never ship anything.”

2 The core idea: hypothesis closure

A hypothesis is *closed* when it has all six of: (i) a precise one-sentence statement (one variable, one measurable outcome); (ii) a predicted TRUE shape, written before measurement; (iii) a predicted FALSE shape (the counterfactual — what you would see if the claim is wrong; if you cannot write it, the claim is unfalsifiable and should be discarded); (iv) independent external evidence measured against both shapes; (v) a verdict of WORKING (with named coverage) or NOT WORKING (with a causal reason); (vi) the verdict written down and dated before downstream work depends on it. The default verdict under ambiguity is NOT WORKING: the burden of proof is on the claim. “Probably,” “it seems to,” “partially,” and “we’ll measure that later” are not verdicts.

This is Popperian falsification [3] applied operationally to ML claims, combined with *pre-registration* — committing the success criteria before seeing results, a practice imported from clinical trials and psychology to combat post-hoc rationalization, and increasingly discussed for ML [4]. The novelty here is not the principle but the operational packaging: a date-locked threshold file, a planted-signal test of the evaluation harness itself, and a closed-loop diagnostic (below).

3 The ten-stage lifecycle

Falsification of the headline claim is **Stage 5**. The full set:

#	Stage	Failure it catches	Tool
0	Frame the problem	building the wrong thing; using ML where a rule suffices	—
1	Data contract + leakage	train/test contamination	leakage
1.5	Data reality	signal/precision the data cannot physically support	reality
2	Baseline before training	inventing the bar after seeing the result	—
2.5	Value / economics	scientifically sound but economically pointless	value
3	Training health	a model converged to noise	—
4	Eval design	a mis-chosen metric; underpowered evaluation	—
5	Falsify the claim	a claim that fits in-sample while measuring nothing real	planted, closed-loop, prereg
6	Deploy + monitor	silent drift after deployment	—
7	Stopping rule	iterating forever; never revisiting a stale claim	—

Each stage emits **PASS** / **FAIL** / **UNKNOWN**. The roll-up rule is mechanical: any FAIL on a critical stage (0, 1, 1.5, 2.5, 5) → **DO NOT PURSUE**; else any UNKNOWN on a critical stage → **INSUFFICIENT EVIDENCE**; all critical PASS → **PURSUE**. The idea-only path (no data yet) leaves data-dependent stages UNKNOWN and typically yields INSUFFICIENT EVIDENCE with a concrete “collect a pilot dataset next” — but can yield DO NOT PURSUE outright when a judgment or feasibility stage fails (e.g. a precision target above the label-noise ceiling).

4 The diagnostics

Three stages have the most automation leverage; the tool implements them as task-agnostic, pure-stdlib CLIs (no numpy/scipy/pandas), supporting classification (AUROC), regression (Pearson), and ranking (Spearman) via a `-metric` flag.

Leakage / split hygiene (leakage, Stage 1). Detects duplicate/near-duplicate rows spanning the train/test split, temporal overlap (test not strictly after train), and group/entity leakage (an entity in both splits) as hard failures; a single feature that perfectly partitions the label is flagged as a *suspect for human review*, not an auto-failure (legitimate downstream-consequence features exist).

Data reality / feasibility (reality, Stage 1.5). In *numbers mode* (no data yet) it computes the minimum detectable correlation at a given sample size via the Fisher-z transform, and the achievable-precision ceiling implied by a stated label-noise rate ($\text{ceiling} = 1 - \text{error_rate}$), returning a verdict on whether the target is reachable *in principle*. In *CSV mode* it reports sample size, class balance, per-column missingness, and label-noise from a re-labeled gold subset. The inverse-normal quantiles use Acklam’s approximation; we verified the power computation is bit-accurate against `scipy.stats` at authoring time (e.g. $n=20 \rightarrow$ minimum detectable $r \approx 0.591$; $n=100 \rightarrow 0.277$).

Value / economics (value, Stage 2.5). Computes gross gain, annual net, and payback period from value-per-unit-of-improvement, expected lift, deploy cost, and maintenance cost, returning PASS / FAIL / UNKNOWN. This catches the “1% lift, \$2M to deploy” project that is scientifically fine and economically dead.

Falsification harness (Stage 5). Two diagnostics support the trust gate. The *planted-signal meta-falsification* (**planted**) injects an obvious signal and confirms the chosen metric is near-perfect, then injects pure noise and confirms the metric collapses to its no-skill floor — a test of *the evaluation machinery itself* before any verdict from it is trusted. The *closed-loop / external-oracle diagnostic* (**closed-loop**) correlates the model’s in-sample score against an *independent* oracle on the same cases (joined by ID, never by row order), with a percentile-bootstrap confidence interval, and flags the signature of a closed loop: high in-sample fit paired with near-zero external correlation. A pre-registration utility (**prereg**) date-locks the TRUE/FALSE shapes and thresholds with a content hash so post-hoc threshold-softening is visible.

The tool ships with a test suite (119 tests at time of writing) in which statistical kernels are checked against hand-derived or scipy-cross-checked values, and each test documents the code mutation it is designed to catch.

5 Illustrative use

We give two short illustrative walkthroughs (not an evaluation — see Section 7).

Idea-only, no data — employee-resignation prediction. A practitioner wants $\geq 90\%$ precision but estimates labels are $\sim 15\%$ noisy. `reality -label-error-rate 0.15 -required-precision 0.90` returns DO NOT PURSUE: the precision ceiling is 0.85, below the requirement, unreachable regardless of model quality — even though statistical power (a 150-case pilot can detect $r \geq 0.3$) and the economics both pass. The decision is reached before any data is collected.

Data-stage — a churn model with a planted leak. On a synthetic 480-row dataset where 30 customers appear in both train and test, `leakage -group-col customer_id` flags the group leak and returns FAIL (exit non-zero), while **closed-loop** against an independent human-label oracle shows the score *does* carry real signal ($\rho \approx 0.37$, CI clears zero). The composed verdict is DO NOT PURSUE *until the split is fixed*: the signal is plausibly real, but the headline metric a stakeholder would see is inflated by the leak — a fixable PURSUE-candidate, not a dead project.

6 Relation to prior work

Each ingredient has antecedents; the contribution is the synthesis and the pre-build framing.

- **Reproducibility & leakage.** Kapoor and Narayanan [1] establish the scale and mechanism of leakage-driven irreproducibility. falsify-ml’s Stage 1 operationalizes a subset of these checks as a runnable pre-training gate.
- **Reporting standards.** REFORMS [2] is the closest relative: a consensus checklist for *reporting* ML-based science. The differences are purpose and timing. REFORMS asks “what should a finished study disclose so a reviewer can judge it?”; falsify-ml asks “should this project be pursued at all, and does its central claim survive an adversarial attack?” — applied *before* and *during*

the work, with runnable diagnostics and a go/no-go verdict rather than a disclosure checklist. The two are complementary.

- **Falsification & pre-registration.** Popper [3] supplies the epistemic stance; pre-registration for ML [4] supplies the commit-criteria-first mechanic. falsify-ml packages both operationally.
- **Power and statistics.** The data-feasibility stage applies standard power analysis and a simple label-noise precision ceiling; these are textbook, not novel — the contribution is making them a *day-zero gate* rather than a post-hoc rationalization.

To our knowledge, the specific packaging — a single pre-build-to-deployment lifecycle that (a) treats the project-level “is this worth pursuing?” question as the top-level output, (b) ships runnable, dependency-free diagnostics for the highest-leverage stages, and (c) works on a bare idea before data exists — is not directly covered by the above. We make this as a *positioning* claim, not a priority claim; we have not done an exhaustive literature survey and welcome pointers to closer work.

7 Limitations and what this paper does *not* claim

In keeping with the protocol’s own discipline, we state the boundaries explicitly.

1. **No empirical evaluation of the protocol’s efficacy.** We have not measured how often falsify-ml would have flagged real, independently-documented ML failures (e.g. the retracted/corrected papers catalogued in [1]). The illustrative cases in Section 5 are anecdotes and one is synthetic. The honest verdict on “this protocol catches real failures at rate X ” is **INSUFFICIENT EVIDENCE**; a retrospective study on a corpus of known-leaky papers is the natural next step.
2. **No adoption or usability data.** Whether practitioners find the protocol usable and act on its verdicts is untested.
3. **The estimates are only as good as their inputs.** The feasibility stage computes correct statistics on *user-supplied* effect sizes and label-noise rates; a confident-looking DO NOT PURSUE built on a guessed input is itself a hazard.
4. **Scope is ML claims with data, labels, and a metric** — classification, regression, ranking, detection, forecasting. It is *not* an evaluator of LLM/agent/generative quality, which needs different machinery.
5. **The diagnostics cover three of ten stages.** The remaining stages are structured checklists, not automated checks; their reliability depends on the practitioner answering them honestly.

A paper for a falsification tool that overstated its own evidence would be self-refuting. We have tried to hold this paper to the standard the tool enforces.

8 Conclusion

falsify-ml reframes ML validation from “find evidence it works” to “try hard to prove it doesn’t, and let it survive only if it genuinely does,” and extends that stance across the whole project lifecycle so that the cheapest place to kill a doomed project — before data, before compute — is where the decision is made. The tool is open source, dependency-free, and task-agnostic. The protocol’s value is, by its own rules, an open hypothesis until evaluated against a corpus of known failures; that evaluation is the work we invite.

References

- [1] S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. <https://doi.org/10.1016/j.patter.2023.100804>
- [2] S. Kapoor et al. REFORMS: Consensus-based Recommendations for Machine-learning-based Science. *Science Advances*, 2024. arXiv:2308.07832. <https://doi.org/10.1126/sciadv.adk3452>
- [3] K. Popper. *The Logic of Scientific Discovery*. 1959.
- [4] L. Bertinetto, J. F. Henriques, S. Albanie, et al. (eds.). *Proceedings of the NeurIPS 2020/2021 Workshops on Pre-registration in Machine Learning*. PMLR Volumes 148 (2020) and 181 (2021). <https://proceedings.mlr.press/v148/>. See also J. M. Hofman, A. Chatzimparmpas, A. Sharma, D. J. Watts, and J. Hullman, Pre-registration for Predictive Modeling, arXiv:2311.18807, 2023.

This paper is methodology and positioning only. It contains no proprietary model, data, or result. The tool's diagnostics run on the user's own project.